buildPARITY - UGV Beast Cyberwave planning

Sam Wilcock

▼ buildPARITY - UGV Beast Cyberwave planning

- Introduction
- High-level architecture
- Work Package Summary
- Project Timeline (Gantt Chart)
- Work Package 1: Base UGV & edge driver development
- Work Package 2: Sensor fusion & edge autonomy
- Work Package 3: Cyberwave Cloud & communications pipeline
- Work Package 4: Control PC, high-level "missions", and VLA integration
- UGV Autonomous skills library (Mission primitives)
- *Bonus* Work Package 5: Stretch/future goals (Digital twins & co-ordination)

Introduction

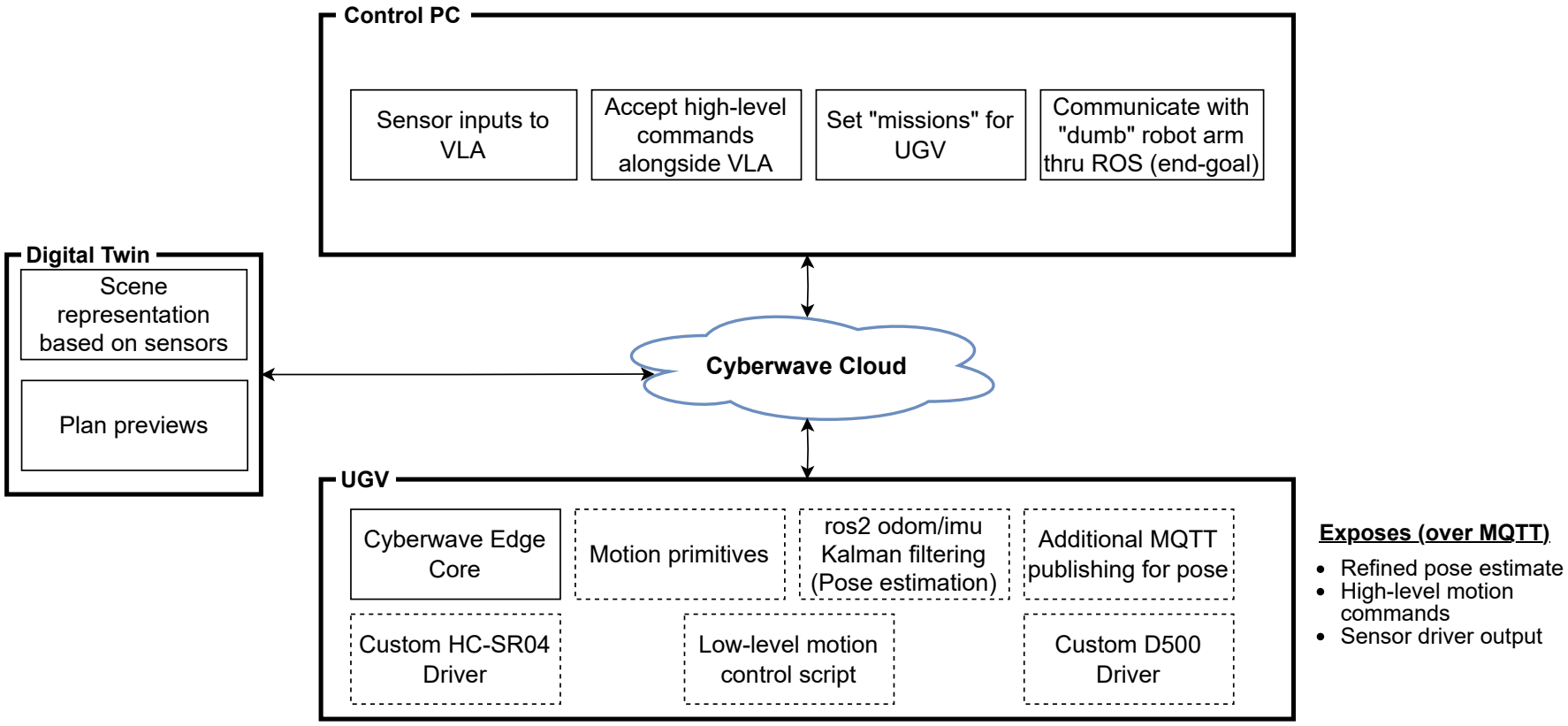
This document outlines the development and integration plan for the UGV Beast project utilising the Cyberwave SDK. The primary objective is to build a semi-autonomous Unmanned Ground Vehicle (UGV) driven by Vision-Language-Action (VLA) models, specifically tailored for **construction environment monitoring**.

The UGV's core use cases are:

1. **Material Tracking:** Navigating the environment to locate and track the position of construction materials (initially using AprilTags).
2. **Hazard Detection:** Continuously monitoring the visual feeds to autonomously identify construction safety hazards using the VLA.

By bridging native ROS2 sensor fusion with the Cyberwave Edge and Cloud infrastructure, the system translates high-level user instructions into actionable, remote "missions" executed by the UGV.

High-level architecture



The system architecture is divided into three distinct operational tiers to balance heavy AI computation with low-latency edge autonomy:

- **Control PC (The Brain):** Interfaces with the user to accept broad commands, contextualizes visual sensor inputs for the VLA pipeline, and dissects AI responses into discrete motion and data-gathering tasks. It dispatches these missions down to the edge and is positioned to eventually coordinate stretch-goal assembly operations with a Fanuc CRX10iA/L arm.
- **Cyberwave Cloud (The Bridge):** Acts as the central communication broker using MQTT and REST APIs. It routes telemetry and video streams from the UGV up to the Control PC, while hosting and routing requests to cloud-side AI models (e.g., Gemini, Nano Banana) and future Digital Twin environments.
- **UGV Beast Layer (The Edge):** Runs on a Raspberry Pi 5 equipped with ROS2 and Cyberwave Edge Core software. To mitigate cloud network latency, high-speed reactions and physical safety overrides remain strictly local. Custom Cyberwave drivers manage hardware like the D500 Lidar, while the edge handles local sensor fusion (via a `ros_navigation` Kalman filter) to push clean, refined pose and telemetric data up to the cloud.

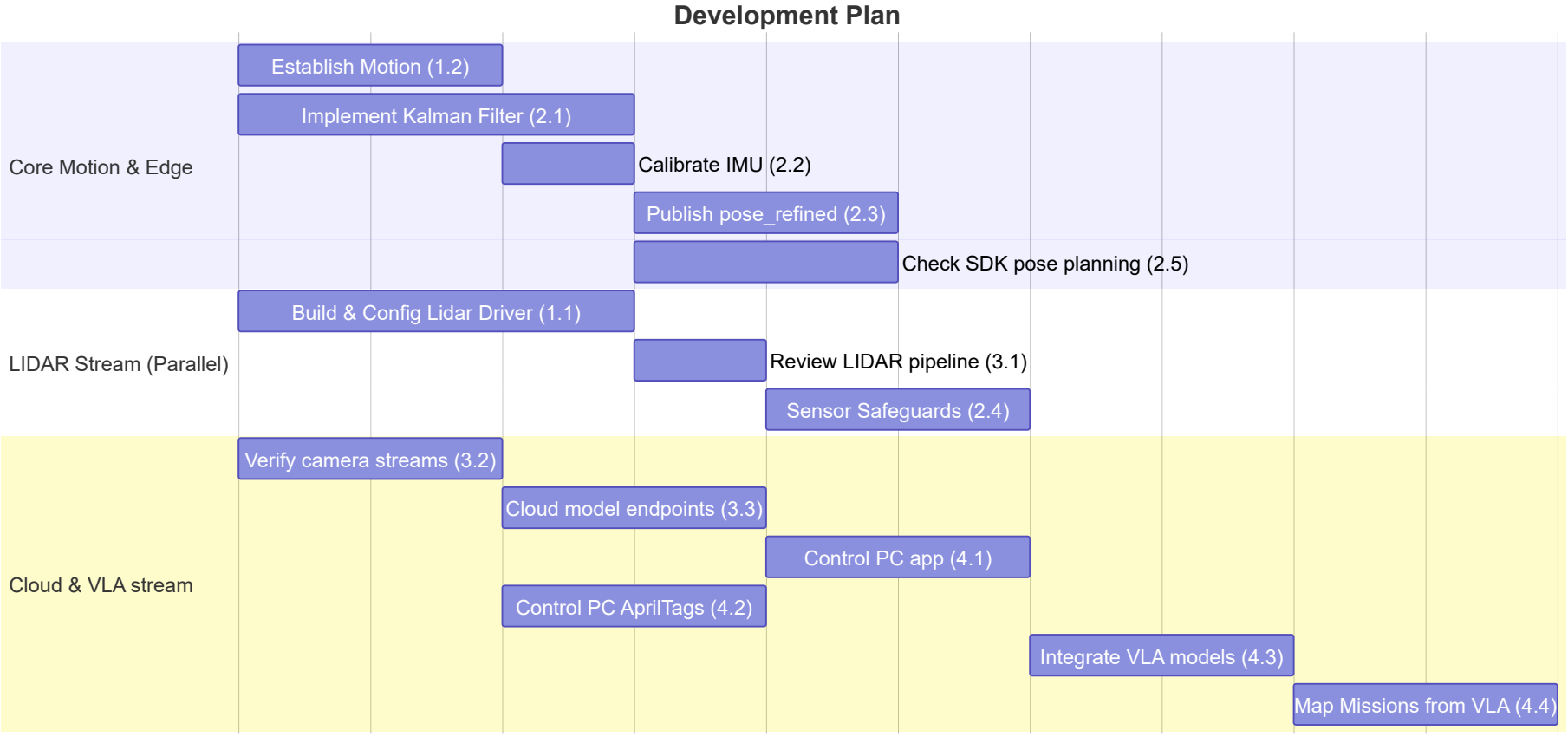
Work Package Summary

Before looking at the timeline overview, here is a quick mapping of the major packages out to their target runtime environments.

ID	Phase / Work Package	Core Deliverables	Target
WP 1	Base UGV & Edge Drivers	Custom D500 Lidar driver (MQTT), base low-latency motion capabilities.	**UGV**
WP 2	Sensor Fusion & Autonomy	Kalman Filter setup, IMU calibration, edge sensor-driven safety loops.	**UGV**
WP 3	Cloud & Comms Pipeline	Lidar & video stream verifications, Cloud AI model endpoint routing.	**Cloud**
WP 4	Control PC & VLA	UI/Chat dev, PC-side AprilTag detection, VLA context & mission mapping.	**PC**
WP 5	Stretch Goals (Future)	Digital Twin mapping, Depth+Lidar fusion, Fanuc Crx arm integration.	**Mixed**

Project Timeline (Gantt Chart)

A projection visualising Work Package dependencies and parallel workflows.



Work Package 1: Base UGV & edge driver development

Since ROS2 and the Cyberwave Edge Core are already installed on the Raspberry Pi 5, the first step is to establish reliable hardware-to-network communication on the edge.

- **Task 1.1:** Build the custom Cyberwave driver for the D500 Lidar using available boilerplate and Claude SDK skills, and configure it to publish raw environmental data over Cyberwave's MQTT bridge.
  - *Subtask 1.1.1:* Define the MQTT topic structure and payload format (e.g., JSON/Protobuf 2D ranges).
  - *Subtask 1.1.2:* Validate local publishing frequency and performance hit on the Pi 5. See example params.yaml.
- **Task 1.2:** Establish basic low-latency motion capabilities (move forward x, move to x/y, orientations) refined with IMU/ros_navigation feedback.

Work Package 2: Sensor fusion & edge autonomy

Handling latency-sensitive and high-speed reactions directly on the UGV prior to cloud communication.

- **Task 2.1:** Implement the `ros_navigation` odometry/IMU augmented Kalman filter on the Pi 5.
- **Task 2.2:** Calibrate IMU. Run stationary and rotational calibration routines to find and set IMU biases.
- **Task 2.3:** Program the addition to the MQTT bridge to publish the resulting `pose_refined` topics.
- **Task 2.4:** Establish sensor-driven motion safeguards (e.g., stopping on ultrasonic/lidar obstacles) at the UGV Beast layer.
- **Task 2.5:** Check existing Cyberwave SDK integration with `ros_navigation` for pose-to-pose planning to inform exactly how route planning to poses is setup. Review Cyberwave documentation/boilerplates for Nav2 goal sending and action serving.

Work Package 3: Cyberwave Cloud & communications pipeline

Connecting the Edge layer with the Control layer. Basic MQTT, REST API, and video streaming are already built into the SDK.

- **Task 3.1:** Review existing MQTT/REST API communications to ensure that the custom LIDAR data is correctly coming through the pipeline with complementary SDK extension functions.
- **Task 3.2:** Verify SDK-provided visual/camera streams from the UGV are routing correctly to the Cloud/Control PC (get_latest_frame).
- **Task 3.3:** Set up endpoints and triggers for cloud-hosted models (Nano Banana, Gemini, etc.).

Work Package 4: Control PC, high-level "missions", and VLA integration

Developing the "Brain" of the operation.

- **Task 4.1:** Develop the Control PC application to accept user high-level commands.
  - *Subtask 4.1.1:* Set up Chat UI workflow to accept instructions (e.g., "Find the red block, Check for existing construction materials").
- **Task 4.2:** Implement AprilTag recognition locally on the Control PC for material tracking.
  - *Subtask 4.2.1:* Ingest the `get_latest_frame` video stream from the Cyberwave Cloud.
  - *Subtask 4.2.2:* Run detection algorithms on the Control PC's CPU/GPU to map construction material locations. This avoids overloading the edge Pi 5, bypassing its slower constraint.
- **Task 4.3:** Integrate the Vision-Language-Action (VLA) models. Feed visual sensor inputs into the VLA
  - *Subtask 4.3.1:* Pass the `get_latest_frame` outputs along with the user prompts into the VLA context window.
  - *Subtask 4.3.2:* Configure standard VLA system prompts targeting the identification of construction materials and flagging of safety hazards in the frame.
- **Task 4.4:** Map VLA outputs and user commands into sets of motion and data-gathering tasks ("Missions") to be sent over the Cyberwave Cloud down to the UGV.
  - *Subtask 4.4.1:* Parse VLA textual/JSON outputs into sequential actionable steps (e.g., Rotate 90 -> Move 2m).

UGV Autonomous skills library (Mission primitives)

To allow the VLA to effectively orchestrate the UGV, the Beast must expose a library of discrete "Skills" (primitives) that can be chained together into missions. Utilizing the chassis movement, 2D Lidar, and the pan-tilt wide-angle camera, the core skills will include:

1. `pan_tilt_sweep` **(Visual Sweep):** Actuates the pan-tilt camera mechanism to capture a comprehensive 360-degree or targeted area view without moving the rover chassis. Used by the VLA for rapid environment hazard scanning or initial material spotting.
2. `targeted_search` **(Active Patrol):** A composite skill where the UGV navigates a search pattern (or rotates) while continuously panning the camera, stopping once the Control PC flags a specific AprilTag or material match.
3. `navigate_to_pose` **(Point-to-Point):** Uses the `ros_navigation` pipeline to drive to an (x, y, theta) coordinate while relying on the native 2D Lidar safety loops to avoid dynamic obstacles.
4. `hazard_overwatch` **(Stationary Monitor):** Parks the UGV at a high-visibility vantage point and sweeps the pan-tilt camera periodically to feed a continuous hazard-detection stream to the Control PC.
5. `center_on_target` **(Visual Servoing):** Continuously adjusts the camera's pan and tilt to keep a recognized AprilTag or VLA bounding box centered in the frame, useful for close-up inspection of materials.

Bonus Work Package 5: Stretch/future goals (Digital twins & co-ordination)

Advanced integrations.

- **Task 5.1:** Feed sensor data and the locally-computed AprilTag poses from the Control PC back up into the Cyberwave Cloud to generate a real-time Digital Twin environment of the UGV and construction objects (predefined meshes?).
- **Task 5.2:** Advanced Markerless Pose Approximation (Depth + Lidar Fusion).
  - *Subtask 5.2.1:* Run DepthAnything vision models on the camera feed and fuse the output with 2D Lidar scans (radar-like sweeps) to approximate material poses/ground-truth without relying on AprilTags.
- **Task 5.3:** Create a custom Cyberwave driver for the HC-SR04 Ultrasonic distance finder.
- **Task 5.4:** Build communication pathways from the Control PC visual input processing to the Fanuc CRX10iA/L arm to orchestrate assembly operations.
  - *Subtask 5.4.1:* Map spatial coordinates gathered by the UGV to the Fanuc's base coordinate system.
- **Task 5.5:** Integrate the Orbbec Astra camera later for pointcloud data?